

Improved Cordganizer Project

Documentation

This project consists of the automated manufacturing and assembly of two components (a lid and a body) using CAD and CAM in Fusion Autodesk, followed by a vision algorithm for quality assurance.

Zip File Structure

File Name	Type	Description
Lid.f3z	CAD	3D Model of lid
Body.f3z	CAD	3D Model of body
Lid.nc	CAM	Milling instructions
Body.nc	CAM	Milling instructions
bodydrawing	CAD Drawing	Dimensioned drawing of Body
liddrawing	CAD Drawing	Dimensioned drawing of Lid
Flipfocus2, listen_final	Python	Python Vision Code Integration

Bill of Materials

This process only requires 2 W stocks and 2 Z stocks. These stocks are loaded into Edie and Rosie. The two base stocks are loaded into Edie and the two top stocks are loaded into Rosie.

Replication instructions:

0. Prerequisites

- Install Autodesk Fusion 360
- Ensure access to the ADML for either of the CNC milling machines

1. CAD and CAM Setup

- Open the provided CAD files in Autodesk Fusion 360

- Review the (lid and body) to understand the design intent
- Navigate to the CAM workspace and load the corresponding toolpaths
- Simulate the toolpaths to verify machining operations, including tool movements and tool changes

2. NC File Export and CNC Preparation

- Post-process the CAM setup to generate NC files
- Export the NC files to the respective CNC milling machine usb
- Copy the program and load it on “Memory” of the CNC.
- When ready start NC Program.

3. Robot and BUMES Preparation

- Prepare all robots, linear sliders, and conveyor belts.
- Open BUMES with the program listed below, and when ready, run program.

New Design

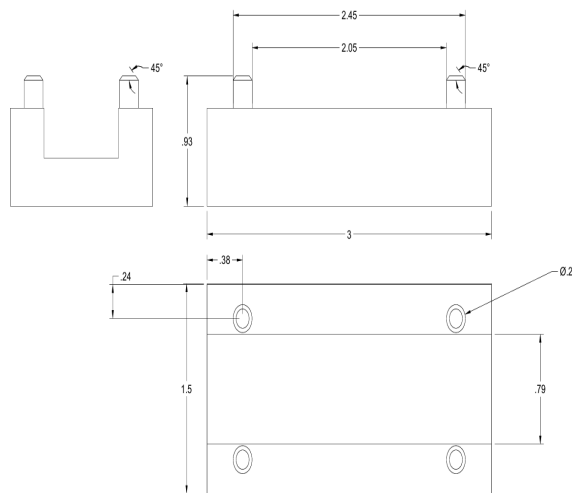


Figure Body Dimensions

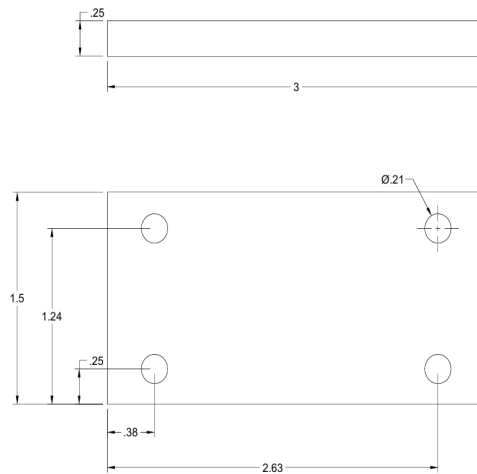


Figure Lid Dimensions

The design for this project was driven by limitations in the original cordganizer, which had long machining times due to complex geometry and limited capacity for holding multiple cords. In our first iteration, we focused on simplifying the design while exploring several fastener configurations and multiple variations of the central channel to improve both functionality and manufacturability.

The gap between the peg and hole was iterated numerous times in order to ensure a tight but reasonable fit for assembly. Final cad dimensions for the hole on the lid measured 0.021” in diameter and the peg on the body measured 0.0195” in diameter, leaving the final part with a radial gap of 0.00075” at the joinery. This fit was the most consistent in ensuring the assembled product would stay together under light loads but still could be assembled and taken apart.

Through iterative refinement, we reduced the design to require only a single tool change, significantly decreasing machining time. Additionally, we adjusted tolerances across all components to improve ease of assembly and overall fit. The final design reflects these improvements and is included in the accompanying ZIP file.

BUMES

Our overall goal with BUMES was to increase throughput by decreasing downtime. We did this by implementing a completely different program structure allowing for parallel processing. In total we used 7 different BUMES programs A-G. As a high level overview we have 7 programs, with 3 programs for each set of parts and 1 for pallet motion. Programs A and C load the lids to be machined as well as unload both mills and move finished parts to assemble. Simultaneously programs E, F and G are running. E and F are responsible for loading and machining the bodies into Edie while G moves all empty pallets from station 1 to the conveyor to begin their travel. Programs B and D, which run once A and C are completed

respectively, are responsible for Assembly and Vision. This program structure along with decreasing machining time and optimizing robot motion allowed us to create 2 parts in 10.8mins, almost 7 minutes faster than the full Cordganizer with additional functionality. Below we have attached CIM tables, BUMES programs and a routing diagram.

CIM Tables:

Process A								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time	Manufacturing Operation Time (Secs)	Utilization
1	Initialize System	Initialize A and Holding program C, waiting for set to manufacture	startUpTasksComplete()	1777561813	1777561813	0.0	0.0	0
			resourceSeize('A')	1777561813	1777561813	0.0		
			resourceSeize('CPrime')	1777561814	1777561814	0.0		
			resourceSeize('Mary')	1777561814	1777561814	0.0		
2	Manufacture Lid	Collect raw lid stock and load Cayenne	urDashboard('Mary', 'me345_admin/gravityFeederW.ulp')	1777561815	1777561850	35.4	69.8	1
			urDashboard('Mary', 'THU_9AM_adminCG-WXYZLoadMILMakeLid_1.ulp')	1777561851	1777561885	34.4		
			resourceSeize('APrime')	1777561885	1777561885	0.0		
			resourceRelease('APrime')	1777561886	1777561886	0.0		
3	Wait for Corresponding Body	Ensure body has started machining	resourceSeize('Edie')	1777561886	1777561886	0.0	48.9	0.00
			resourceSeize('Rosie')	1777561887	1777561936	48.9		
			resourceSeize('convStn4', 'agvD')	1777561936	1777561955	31.9		
			resourceSeize('convStn3', 'agvA')	1777561956	1777561992	23.9		
4	Retrive Empty Pallets	Move empty pallets on conveyor to correct station	urDashboard('Mary', 'me345_admin_adminCG-WXYZUnloadMILid.ulp')	1777561992	1777562015	22.2	55.8	0.80
			urDashboard('Mary', 'THU_9AM_adminCG-PhometsBelt.ulp')	1777562015	1777562038	22.9		
			resourceRelease('Mary')	1777562038	1777562038	0.0		
			urDashboard('Edie', 'me345_admin_adminCG-WXYZUnloadMILBody.ulp')	1777562039	1777562061	21.9		
5	Unload Lid	Collect finished lid and place on conveyor	urDashboard('Edie', 'THU_9AM_adminCG-WXYZToConv_DIRECT.ulp')	1777562061	1777562075	14.6	45.2	0.05
			resourceRelease('Edie')	1777562075	1777562076	0.0		
			resourceRelease('CPrime')	1777562076	1777562076	0.0		
			resourceRelease('convStn4')	1777562077	1777562077	0.0		
6	Unload Body	Collect finished body and place on conveyor	resourceSeize('convStn1', 'agvD')	1777562077	1777562091	13.3	36.5	0.52
			resourceRelease('convStn3')	1777562091	1777562091	0.0		
			urDashboard('Rosie', 'me345_admin_adminCG-ConveyorToPhome.ulp')	1777562092	1777562103	11.9		
			urDashboard('Rosie', 'me345_admin_adminCG-PhomeToInB.ulp')	1777562104	1777562115	11.3		
7	System Handshake	First set of parts done machining second set can start now	resourceRelease('convStn1')	1777562115	1777562115	0.0	45.2	0.05
			resourceSeize('convStm1', 'agvA')	1777562116	1777562116	2.2		
			urDashboard('Rosie', 'me345_admin_adminCG-ConveyorToPhome.ulp')	1777562119	1777562129	11.4		
			urDashboard('Rosie', 'me345_admin_adminCG-PhomeToInA.ulp')	1777562130	1777562141	11.2		
8	Finished Parts to Station 1	Bring finished objects to INVA and INVB at Station 1 to prepare for Assembly	resourceRelease('Rosie')	1777562141	1777562141	0.0	61.2	0.88
			resourceRelease('convStn1')	1777562142	1777562142	0.0		
			resourceRelease('A')	1777562143	1777562143	0.0		
			endProcess()	1777562143	1777562143	0.0		

Process G								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time (secs)	Manufacturing Operation Time (Secs)	Utilization
1	Manufacture Body	Place all pallets on AGVs sequentially	startUpTasksComplete()	1777561825	1777561825	0.0	107.4	1
			resourceSeize('convStn1', 'agvA')	1777561826	1777561854	28.3		
			resourceSeize('Rosie')	1777561854	1777561854	0.0		
			urDashboard('Rosie', 'me345_admin_adminCG-invAToPhome.ulp')	1777561855	1777561868	13.2		
			urDashboard('Rosie', 'me345_admin_adminCG-PhomeToConveyor.ulp')	1777561868	1777561882	13.6		
			urDashboard('Rosie', 'me345_admin_adminCG-invBToPhome.ulp')	1777561882	1777561892	10.3		
			urDashboard('Rosie', 'THU_9AM/invBtoagvB.ulp')	1777561892	1777561901	8.3		
			urDashboard('Rosie', 'me345_admin_adminCG-invCToPhome.ulp')	1777561901	1777561908	7.1		
			urDashboard('Rosie', 'THU_9AM/invCtoagvC.ulp')	1777561908	1777561917	9.2		
			urDashboard('Rosie', 'me345_admin_adminCG-invDToPhome.ulp')	1777561918	1777561925	7.1		
			urDashboard('Rosie', 'THU_9AM/invDtoagvD.ulp')	1777561925	1777561935	10.2		
			resourceRelease('convStn1')	1777561935	1777561935	0.0		
			resourceRelease('Rosie')	1777561936	1777561936	0.0		
			endProcess()	1777561936	1777561936	0.0		

Process E								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time (secs)	Manufacturing Operation Time (Secs)	Utilization
1	Manufacture Body	Collect raw stock and load Cayenne	startUpTasksComplete()	1777561821	1777561821	0.0	51.0	1
			resourceSeize('APrime')	1777561821	1777561821	0.0		
			resourceSeize('Edie')	1777561822	1777561822	0.0		
			urDashboard('Edie', 'THU_9AM/')	1777561822	1777561837	14.1		
			urDashboard('Edie', 'THU_9AM/')	1777561837	1777561874	36.9		
			resourceRelease('Edie')	1777561874	1777561874	0.0		
			resourceRelease('APrime')	1777561874	1777561874	0.0		
			endProcess()	1777561875	1777561875	0.0		

Process B								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time	Manufacturing Operation Time (Secs)	Utilization
1	System Handshake	Wait for lid and body to be done	startUpTasksComplete()	1777561815	1777561815	0.0	327.3	1
			resourceSeize('B')	1777561815	1777561815	0.0		
			resourceSeize('A')	1777561816	1777562143	327.3		
			resourceRelease('A')	1777562144	1777562144	0.0		
2	Assembly	Assemble Lid to body	resourceSeize('Rosie')	1777562144	1777562144	0.0	82.0	0.25
			urDashboard('Rosie', 'THU_9AM/Thursday9_ABassembly.ulp')	1777562145	1777562227	82.0		
			urDashboard('Rosie', 'THU_9AM/Thursday9_visIntegrb.ulp')	1777562227	1777562252	25.0		
			resourceRelease('Rosie')	1777562252	1777562252	0.0		
3	Vision Test	Divide parts into pass and fail	resourceRelease('B')	1777562252	1777562252	0.0	25.0	0.08
			endProcess()	1777562253	1777562253	0.0		

Process C								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time	Maufacturing Operation Time (Secs)	Utilization
1	Intialize System	Ensuring first set has finished machining	startupTasksComplete()	1777561817	1777561817	0.0	259.5	1.00
			resourceSeize('C')	1777561817	1777561817	0.0		
			resourceSeize('CPrime')	1777561818	1777562077	259.5		
			resourceRelease('CPrime')	1777562078	1777562078	0.0		
2	Manufacture Lid	Collect raw stock and load cayenne	resourceSeize('Mary')	1777562078	1777562078	0.0	69.5	0.27
			urDashboard('Mary','me345_admin/gra	1777562079	1777562115	36.1		
3	Retrive empty pallets	Move empty pallets to correct station	urDashboard('Mary','THU_9AM/_admin	1777562115	1777562148	33.4	51.5	0.20
			resourceSeize('convStn4','agvC')	1777562148	1777562174	25.5		
4	Wait for Corresponding Body	Ensure Body has started maching	resourceSeize('convStn3','agvB')	1777562175	1777562201	26.0	0.0	0.00
			resourceSeize('BPrime')	1777562201	1777562201	0.0		
			resourceRelease('BPrime')	1777562202	1777562202	0.0		
			resourceSeize('Edie')	1777562202	1777562202	0.0		
5	Unload Lid	Collect finished lid and place on conveyor	urDashboard('Mary','me345_admin/_ad	1777562203	1777562237	34.5	57.3	0.22
			urDashboard('Mary','THU_9AM/_admin	1777562237	1777562280	22.8		
			resourceRelease('Mary')	1777562281	1777562281	0.0		
6	Unload Body	Collect finished body and place on conveyor	urDashboard('Edie','me345_admin/_ad	1777562281	1777562284	23.1	23.1	0.09
			urDashboard('Edie','THU_9AM/_admin	1777562285	1777562299	14.4		
			resourceRelease('Edie')	1777562299	1777562299	0.0		
			resourceRelease('convStn4')	1777562300	1777562300	0.0		
			resourceRelease('convStn3')	1777562300	1777562300	0.0		
			resourceSeize('convStn1','agvC')	1777562301	1777562313	12.6		
			resourceSeize('B')	1777562314	1777562314	0.0		
			resourceRelease('B')	1777562314	1777562314	0.0		
			resourceSeize('Rosie')	1777562315	1777562315	0.0		
			urDashboard('Rosie','me345_admin/_a	1777562316	1777562327	11.7		
			urDashboard('Rosie','me345_admin/_a	1777562327	1777562337	9.2		
			resourceRelease('convStn1')	1777562337	1777562337	0.0		
			resourceSeize('convStn1','agvB')	1777562337	1777562343	5.0		
			urDashboard('Rosie','me345_admin/_a	1777562343	1777562354	11.9		
			urDashboard('Rosie','me345_admin/_a	1777562355	1777562365	10.1		
			resourceRelease('Rosie')	1777562365	1777562365	0.0		
			resourceRelease('convStn1')	1777562366	1777562366	0.0		
resourceRelease('C')	1777562367	1777562367	0.0					
7	Finished Parts to station 1	Bring finished objects to INVC and INVD at station 1 to prepare for Assembly	endProcess()	1777562367	1777562367	0.0	60.6	0.23

Process F								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time (Secs)	Manufacturing Operation Time (Secs)	Utilization
1	System Handshake	Wait for program C to start before loading	startupTasksComplete()	1777561823	1777561823	0.0	252.7	1
			resourceSeize('CPrime')	1777561824	1777562076	252.7		
			resourceRelease('CPrime')	1777562077	1777562077	0.0		
			resourceSeize('BPrime',360)	1777562077	1777562077	0.0		
2	Manufacture Second Body	Collect raw material and load Paprika	resourceSeize('Edie')	1777562078	1777562078	0.0	50.9	0.20
			urDashboard('Edie','THU_9AM/gravilyFeederZ_1.urp')	1777562078	1777562092	13.5		
			urDashboard('Edie','THU_9AM/_adminCG-WXYZLoadMillMakeBody2.urp')	1777562092	1777562130	37.4		
			resourceRelease('Edie')	1777562130	1777562130	0.0		
			resourceRelease('BPrime')	1777562130	1777562130	0.0		
			endProcess()	1777562131	1777562131	0.0		

Process D								
Manufacturing Operation Number	Manufacturing Operation Label	Manufacturing Command	Task Command	Start Time	End Time	Task Time (Secs)	Manufacturing Operation Time (Secs)	Utilization
1	System Handshake	Wait for lid and body to be done and at station 1	startupTasksComplete()	1777561819	1777561819	0.0	547.5	1.00
			resourceSeize('C')	1777561819	1777562367	547.5		
			resourceRelease('C')	1777562367	1777562367	0.0		
			resourceSeize('D')	1777562368	1777562368	0.0		
2	Assembly	Assemble Lid to body	resourceSeize('Rosie')	1777562368	1777562368	0.0	82.0	0.15
			urDashboard('Rosie','THU_9AM/Thursday0_CDassembly_new.urp')	1777562369	1777562451	81.9		
3	Vision Test	Divide parts into pass and fail piles based on specified tolerance	urDashboard('Rosie','THU_9AM/Thursday0_visintegvb.urp')	1777562451	1777562475	24.0	24.0	0.04
			resourceRelease('Rosie')	1777562475	1777562475	0.0		
			resourceRelease('D')	1777562475	1777562475	0.0		
			endProcess()	1777562476	1777562476	0.0		

Programs:

A

```
//NewA Program
```

```
//Mary robot, lid manufacturing. Lids preloaded in grav holder
```

```
resourceSeize('A')
```

```
resourceSeize('CPrime') //all mills are unloaded, can load new stock
```

```
//Collect raw material and manufacture on Mary
```

//E should be going simultaneously to load Edie

```
resourceSeize('Mary') //Load top onto A
urDashboard('Mary', 'THU_9AM/gravityFeederW_3.urp') //NEW gravity feeder program efficient
urDashboard('Mary','THU_9AM/_adminCG-WXYZLoadMillMakeLid_1.urp')
```

```
resourceSeize('APrime') // waits for E to be done milling before starting next task
resourceRelease('APrime') //releases E again. Handshake stop point.
resourceSeize('Edie') //so C doesn't take over. add time to C
resourceSeize('Rosie')
```

```
//Grab pallet A and B
//Retrieve Empty Pallet from Conveyor at Station 3
resourceSeize('convStn4','agvD') //D goes to Edie
//resourceSeize('convStn2','agvB') //BC stationary by Cayenne
resourceSeize('convStn3','agvA') //A gets pulled to Mary
```

```
//Unload the mill
urDashboard('Mary','me345_admin/_adminCG-WXYZUnloadMillLid.urp')
urDashboard('Mary','THU_9AM/_adminCG-PHometoBelt.urp')
resourceRelease('Mary')
urDashboard('Edie', 'me345_admin/_adminCG-WXYZUnloadMillBody.urp')
urDashboard('Edie', 'THU_9AM/_adminCG-WXYZToConv_DIRECT.urp')
resourceRelease('Edie')
```

```
//mills are empty, robots are released
resourceRelease('CPrime')
```

```
resourceRelease('convStn4')
resourceSeize('convStn1','agvD') //D releases and goes to Rosie
resourceRelease('convStn3') //D and then A are waiting at convStn1
urDashboard('Rosie','me345_admin/_adminCG-ConveyorToPHome.urp')
urDashboard('Rosie', 'me345_admin/_adminCG-PHomeToInvB.urp') //pull D off the line to slot B
```

```
resourceRelease('convStn1')
resourceSeize('convStn1','agvA')
urDashboard('Rosie','me345_admin/_adminCG-ConveyorToPHome.urp')
urDashboard('Rosie', 'me345_admin/_adminCG-PHomeToInvA.urp') //pull A off the line to slot A
```

```
resourceRelease('Rosie')
resourceRelease('convStn1') //Mary part (lid) is in A, Edie part (base) is in B

resourceRelease('A')

//CALL ASSEMBLY PROGRAM
```

B

```
//B Program

//Assembly 1 Program. Stocks are ready at Mary, needs to assemble and then
// reset pallets for next function.

// add start up tasks complete so we can start B and C simult
// D waits for C to end
// C doesn't need pallet loading tasks that are done in A.

resourceSeize('B')

resourceSeize('A')
resourceRelease('A')
resourceSeize('Rosie')

urDashboard('Rosie','THU_9AM/Thursday9_ABassembly.urp')
urDashboard('Rosie','THU_9AM/Thursday9_visIntegvb.urp')
//resourceRelease('Rosie')

resourceRelease('B')
```

C

```
//New C Program

//One part is already finished.
//Same program as ATestv1 but need to change variables to utilize B and C
//pallets instead of A and D respectively.
resourceSeize('C') //waits 3 min before trying to grab Mary
```

```
//resourceSeize('A') //waits for A to finish in order to run
resourceSeize('CPrime') // waits for mills to be empty to load new stock
resourceRelease('CPrime')
```

```
//Collect raw material and manufacture on Mary
//F should be going simultaneously
```

```
resourceSeize('Mary') //Load top onto A
urDashboard('Mary', 'THU_9AM/gravityFeederW_3.urp') //NEW gravity feeder program efficient
urDashboard('Mary', 'THU_9AM/_adminCG-WXYZLoadMillMakeLid_1.urp')
//CHANGE CNC program its calling
```

```
//resourceRelease('Mary')
```

```
//Grab pallet B and C now that they are loaded
//Retrieve Empty Pallet from Conveyor at Station 3
resourceSeize('convStn4','agvC') //C goes to Edie body
resourceSeize('convStn3','agvB') //B gets pulled to Mary lid
```

```
resourceSeize('BPrime') // waits for E to be done milling before starting next task
resourceRelease('BPrime') //releases E again. Handshake stop point.
resourceSeize('Edie')
```

```
//Unload the mill
urDashboard('Mary', 'me345_admin/_adminCG-WXYZUnloadMillLid.urp')
urDashboard('Mary', 'THU_9AM/_adminCG-PHometoBelt.urp')
```

```
resourceRelease('Mary')
```

```
urDashboard('Edie', 'me345_admin/_adminCG-WXYZUnloadMillBody.urp')
urDashboard('Edie', 'THU_9AM/_adminCG-WXYZToConv_DIRECT.urp')
```

```
resourceRelease('Edie')
resourceRelease('convStn4')
resourceRelease('convStn3')
resourceSeize('convStn1','agvC') //C releases and goes to Rosie
```



```
resourceSeize('B') //waits for pallets to be put back onto conveyor from spots
//A and B before sending to respective stations
resourceRelease('B')
```

```
//body at invD=agvC, lid at invC=agvB
```

```
resourceSeize('Rosie')
urDashboard('Rosie','me345_admin/_adminCG-ConveyorToPHome.urp')
urDashboard('Rosie', 'me345_admin/_adminCG-PHomeToInvD.urp') //pull C off the line to slot B
```

```
resourceRelease('convStn1')
resourceSeize('convStn1','agvB')
urDashboard('Rosie','me345_admin/_adminCG-ConveyorToPHome.urp')
urDashboard('Rosie', 'me345_admin/_adminCG-PHomeToInvC.urp') //pull B off the line to slot A
```

```
resourceRelease('Rosie')
resourceRelease('convStn1') //Mary part (lid) is in A, Edie part (base) is in B
```

```
resourceRelease('C') //allows D to run after
```

```
//CALL ASSEMBLY PROGRAM
```

```
// add start up tasks complete so we can start B and C simult
// D waits for C to end
// C doesn't need pallet loading tasks that are done in A.
```

D

```
//D: Assembly 1 Program. Stocks are ready at Mary, needs to assemble and then
// reset pallets for next function.
```

```
// add start up tasks complete so we can start B and C simult
// D waits for C to end
// C doesn't need pallet loading tasks that are done in A.
```

```
resourceSeize('C')
resourceRelease('C')
```

```
resourceSeize('D')
resourceSeize('Rosie')
urDashboard('Rosie','THU_9AM/Thursday9_CDassembly.urp')
```

```
//change to CD  
urDashboard('Rosie','THU_9AM/Thursday9_visIntegvb.urp')
```

```
resourceRelease('Rosie')
```

```
resourceRelease('D')
```

E

```
//E Program  
//Just Load Mary
```

```
resourceSeize('APrime')
```

```
//Collect raw material and manufacture on Edie  
resourceSeize('Edie') //Load bottom onto B  
urDashboard('Edie', 'THU_9AM/gravityFeederZ_1.urp') //UPDATED  
urDashboard('Edie', 'THU_9AM/_adminCG-WXYZLoadMillMakeBody.2.urp')  
//CHANGE CNC program its calling
```

```
resourceRelease('Edie')  
resourceRelease('APrime')
```

F

```
//F program  
//Just Load Edie
```

```
resourceSeize('CPrime') // waits for mills to be empty to load new stock  
resourceRelease('CPrime')
```

```
resourceSeize('BPrime',360) //waits 3 min before trying to grab Edie
```

```
//Collect raw material and manufacture on Edie  
resourceSeize('Edie') //Load bottom onto B  
urDashboard('Edie', 'THU_9AM/gravityFeederZ_1.urp')  
urDashboard('Edie', 'THU_9AM/_adminCG-WXYZLoadMillMakeBody.2.urp')
```

```
resourceRelease('Edie')  
resourceRelease('BPrime')
```

G

//Load all pallets onto conveyor

//Move Empty Pallet to Conveyor at Station 1

resourceSeize('convStn1','agvA')

resourceSeize('Rosie')

urDashboard('Rosie','me345_admin/_adminCG-invAToPHome.urp')

urDashboard('Rosie','me345_admin/_adminCG-PHomeToConveyor.urp')

urDashboard('Rosie','me345_admin/_adminCG-invBToPHome.urp')

urDashboard('Rosie','THU_9AM/invBtoagvB.urp')

//CHANGE! to move directly to next tray B (NP)

urDashboard('Rosie','me345_admin/_adminCG-invCToPHome.urp')

urDashboard('Rosie','THU_9AM/invCtoagvC.urp')

//CHANGE! to move directly to next tray C (NP)

urDashboard('Rosie','me345_admin/_adminCG-invDToPHome.urp')

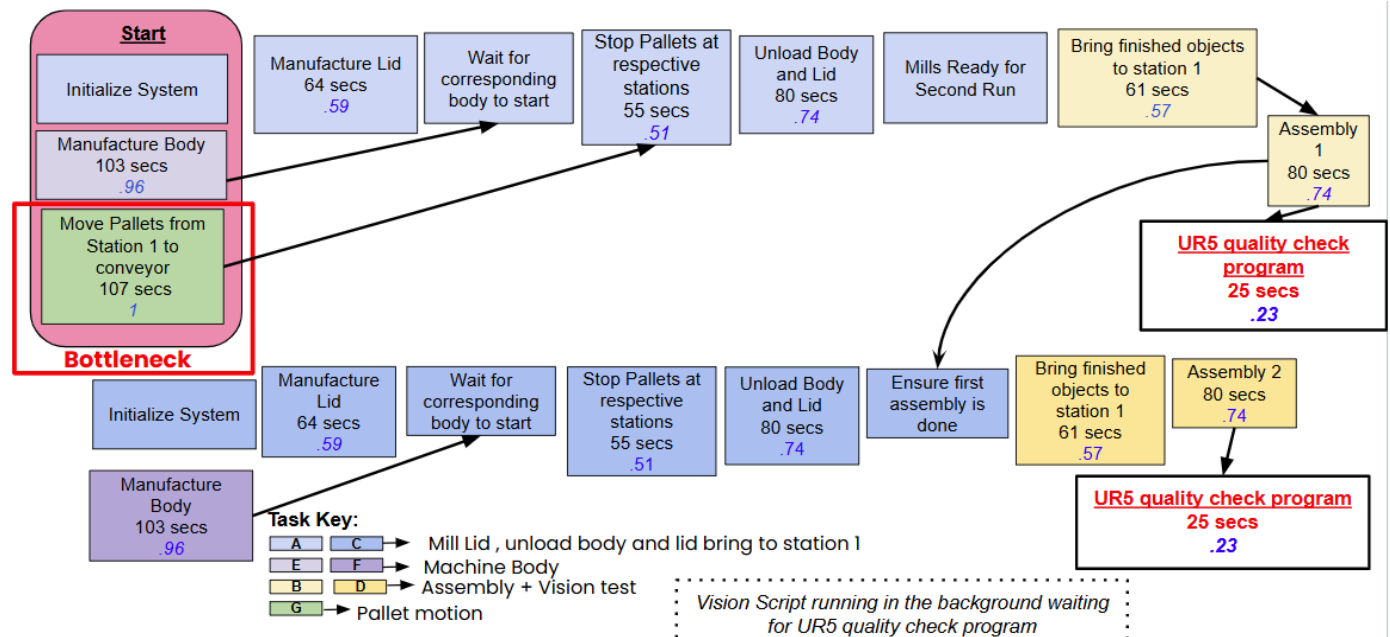
urDashboard('Rosie','THU_9AM/invDtoagvD.urp')

//CHANGE! to move directly to next tray D (NP)

resourceRelease('convStn1')

resourceRelease('Rosie')

Routing Diagram



Vision

Overview

This document contains the descriptions of each python script used for the computer vision integrated quality process of the Cordganizer Optimization. listen_final.py is the file that communicates with the UR5 robot and what relays the vision analysis output back to the robot. flipfocus2.py is the file that captures the photo of the part, runs the vision analysis on the captured image and outputs the results. All python scripts can be found within the folder.

listen_final.py

This file is the bridge between the UR5 robot and the computer running the python scripts. It listens via socket connection over port 29999 on the UR5 robot. Once the robot sends the trigger message “run_my_script” the script changes the value on Modbus 3 of the robot to 0. This acts as a placeholder for the vision analysis result. It then calls the run_edge_detection() function and stores it to a result variable. The pass or fail result is then converted to a value to change the Modbus to. If the result is a pass, it updates the Modbus to 3 and if the result is a fail, it updates it to 4. From that Modbus value, the robot program decides its motion. The script then sets the Modbus back to 0 as a reset then closes the socket connection. This file can be called once and will be run continuously in the background.

```
import socket
import time
from pymodbus.client import ModbusTcpClient
from flipfocus2 import run_edge_detection

# =====
# CONFIG
# =====
HOST = "0.0.0.0"
PORT = 29999
TRIGGER_MESSAGE = "run_my_script"

ROBOT_IP = "10.241.34.45"
MODBUS_PORT = 502
REGISTER_ADDRESS = 129

WAITING_VALUE = 0
PASS_VALUE = 3
FAIL_VALUE = 4

# How long to leave the 3 or 4 on the register before resetting to 0
RESULT_HOLD_TIME = 1

def convert_vision_result_to_shift_value(result):
    """
    Convert the vision result dictionary into the robot decision value.
    """
    if result["passed"]:
        return PASS_VALUE
    return FAIL_VALUE

def update_shift_register(shift_value):
```

```

"""
Connect to the robot over Modbus TCP and write the shift value
into the chosen register.
"""
client = ModbusTcpClient(ROBOT_IP, port=MODBUS_PORT)

try:
    if not client.connect():
        print("Connection to Rosie failed.")
        return False

    response = client.write_register(REGISTER_ADDRESS, shift_value)

    if response.isError():
        print(f"Failed to write {shift_value} to register
{REGISTER_ADDRESS}.")
        return False

    print(f"Successfully sent {shift_value} to MODBUS register
{REGISTER_ADDRESS}.")
    return True

except Exception as e:
    print(f"Modbus write error: {e}")
    return False

finally:
    client.close()

def my_custom_logic():
    """
    Main workflow after the robot sends the trigger:
    1. Reset register to 0
    2. Run edge detection
    3. Convert result into a robot decision value
    4. Write that value to the robot register
    5. Hold briefly so robot can read it
    6. Reset register back to 0
    """
    print("Robot triggered the script.")

    # Reset to 0 before starting
    print("Resetting shift register to 0 before vision run...")
    reset_ok = update_shift_register(WAITING_VALUE)
    if not reset_ok:
        print("Warning: could not reset register to 0 before vision run.")

```

```

# Run vision
print("Running vision...")
result = run_edge_detection()

print(f"Status: {result['status']}")
print(f"Passed: {result['passed']}")

# Convert result to robot value
shift_value = convert_vision_result_to_shift_value(result)
print(f"Shift value to send: {shift_value}")

# Send 3 or 4
write_ok = update_shift_register(shift_value)
if not write_ok:
    print("Warning: failed to write final pass/fail value.")
    return result

print("Vision finished and shift register updated.")

# Hold the result briefly so the robot has time to read it
time.sleep(RESULT_HOLD_TIME)

# Reset back to 0 after sending result
print("Resetting shift register back to 0 after sending result...")
reset_after_ok = update_shift_register(WAITING_VALUE)
if not reset_after_ok:
    print("Warning: failed to reset register to 0 after final
result.")
else:
    print("Shift register reset to 0.")

return result

def handle_connection(conn, addr):
    """
    Handle one robot connection.
    """
    print(f"Connected by {addr}")

    try:
        data = conn.recv(1024).decode("utf-8").strip()
        print(f"Received: {data}")

        if data == TRIGGER_MESSAGE:
            try:

```

```

        my_custom_logic()
    except Exception as e:
        print(f"Vision logic error: {e}")
    else:
        print("Trigger message did not match.")

except Exception as e:
    print(f"Socket handling error: {e}")

finally:
    print("Connection closed.\n")

def start_server():
    """
    Start a TCP server that waits for the robot to connect and send
    the trigger message.
    This keeps running forever until you stop the Python script.
    """
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(1)

    print(f"Server started. Waiting for UR5 on {HOST}:{PORT}...")

    while True:
        try:
            conn, addr = s.accept()
            with conn:
                handle_connection(conn, addr)
        except Exception as e:
            print(f"Server loop error: {e}")

if __name__ == "__main__":
    start_server()

```


flipfocus2.py

This file performs the computer vision inspection that determines whether the part passes or fails alignment requirements. It begins by capturing an image from the webcam and applying the camera focus settings before the image is taken. After capture, the script can optionally save the full image, then applies a light contrast and brightness adjustment to make the part easier to detect. It then crops the image to the area where the part is expected and splits that cropped image into two regions of interest: one for the white body and one for the lid.

The body edges are found first by thresholding the body ROI to isolate the bright white section, then scanning across each valid row to find the first and last white pixels. The script uses the median of those detected edge positions to make the body edge detection more stable. After that, the lid edges are found separately using vertical edge detection. The lid search is limited to a small margin around the measured body edges so the script looks only where the lid edges are expected to appear. The left lid edge is chosen as the leftmost strong candidate in the left search band, and the right lid edge is chosen as the rightmost strong candidate in the right search band.

The script then uses a real-world scale to convert the vision measurements from pixels into inches. It assumes that the white body is 1.5 inches wide in real life, measures that same body width in pixels from the current image, and uses that to calculate pixels per inch and inches per pixel for that specific run. This means the scale is recalculated each time from the measured body width in the captured image, rather than using one fixed conversion value. Using that scale, the script maps the left and right alignment offsets from pixel distances into real physical distances in inches.

The pass/fail logic is based on an engineering tolerance of 0.05 inches. The script converts that inch tolerance into a pixel tolerance using the measured scale from the current image. It then compares the lid-to-body offset on both the left and right sides against that allowable tolerance. If both offsets are within tolerance, the part passes; if either side is outside tolerance, the part fails. The final output is returned as a dictionary containing the detected body and lid edge positions, the measured body width, the calculated scale factors, the offset values in both pixels and inches, the converted pixel tolerance, and the overall pass/fail result. If either the body or lid cannot be detected properly, the script returns a detection error result instead.

```
import cv2
import numpy as np
import time

# =====
# SETTINGS
# =====
```

```

CAMERA_INDEX = 1
WARMUP_TIME = 1.0

# -----
# CAMERA FOCUS SETTINGS
# -----
USE_AUTOFOCUS = False
MANUAL_FOCUS_VALUE = 30
FOCUS_SETTLE_TIME = 0.7
FOCUS_READ_FRAMES = 8

# -----
# REAL-WORLD CALIBRATION
# -----
# The white body width is assumed to be 1.5 inches.
# We measure the body width each run from the captured
# image and compute px/in from that image.
# -----
BODY_REAL_WIDTH_IN = 1.5

# -----
# PASS/FAIL TOLERANCE
# -----
# This is the actual engineering tolerance in inches.
# The code converts it to pixels using the measured
# body width from THIS image.
# -----
ALIGN_TOLERANCE_IN = 0.05

# -----
# CROP
# -----
# Matching testt.py
# -----
CROP_X = 200
CROP_Y = 236
CROP_W = 280
CROP_H = 144

# -----
# BODY ROI
# -----
# Matching testt.py
# -----
BODY_ROI_Y1 = 55
BODY_ROI_Y2 = 100

```

```

# -----
# LID ROI
# -----
# Matching testt.py
# -----
LID_ROI_Y1 = 100
LID_ROI_Y2 = 150

# -----
# BODY DETECTION
# -----
BODY_THRESHOLD = 178
MIN_BODY_ROW_PIXELS = 34

# -----
# IMAGE PREPROCESSING
# -----
CONTRAST_ALPHA = 1.08
BRIGHTNESS_BETA = -18

# -----
# LID DETECTION
# -----
LID_EDGE_THRESHOLD = 20
LID_SEARCH_MARGIN_PX = 26
MIN_LID_COLUMN_SIGNAL = 3

# -----
# OPTIONAL: SAVE CAPTURED IMAGE
# -----
SAVE_CAPTURED_IMAGE = True
CAPTURED_IMAGE_PATH = r"C:/Users/adml-admin/Downloads/image.jpg"

# =====
# FUNCTIONS
# =====

def capture_image_from_webcam(camera_index=1, warmup_time=1.0):
    """
    Captures one image from the webcam.
    Keeps the focus behavior from flipfocus.py
    """
    cap = cv2.VideoCapture(camera_index, cv2.CAP_DSHOW)

    if not cap.isOpened():

```

```

        raise RuntimeError(f"Could not open webcam at index
{camera_index}")

    time.sleep(warmup_time)

    if USE_AUTOFOCUS:
        cap.set(cv2.CAP_PROP_AUTOFOCUS, 1)
    else:
        cap.set(cv2.CAP_PROP_AUTOFOCUS, 0)
        cap.set(cv2.CAP_PROP_FOCUS, MANUAL_FOCUS_VALUE)

    time.sleep(FOCUS_SETTLE_TIME)

    for _ in range(FOCUS_READ_FRAMES):
        ret, frame = cap.read()
        if not ret:
            cap.release()
            raise RuntimeError("Could not capture image from webcam.")

    ret, frame = cap.read()
    cap.release()

    if not ret or frame is None:
        raise RuntimeError("Could not capture image from webcam.")

    return frame


def preprocess(image):
    """
    Light contrast / brightness adjustment.
    Matching testt.py
    """
    return cv2.convertScaleAbs(
        image,
        alpha=CONTRAST_ALPHA,
        beta=BRIGHTNESS_BETA
    )


def detect_body_edges(roi):
    """
    Detects left/right body edges from the white body region.

    Logic:
    - Convert ROI to grayscale
    - Threshold to isolate bright white body

```

```

- For each row, find first and last white pixel
- Use median left/right over valid rows for robustness
"""
gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, BODY_THRESHOLD, 255,
cv2.THRESH_BINARY)

left_candidates = []
right_candidates = []

for row in range(binary.shape[0]):
    pixels = np.where(binary[row] > 0)[0]

    if len(pixels) < MIN_BODY_ROW_PIXELS:
        continue

    left_candidates.append(int(pixels[0]))
    right_candidates.append(int(pixels[-1]))

if not left_candidates:
    return None, None, binary

body_l = int(np.median(left_candidates))
body_r = int(np.median(right_candidates))

return body_l, body_r, binary

def detect_lid_edges(roi, body_l, body_r):
    """
    Detect lid left/right edges from vertical edge strength
    near the body edges.

    Fix:
    - left lid edge: use leftmost strong candidate
    - right lid edge: use rightmost strong candidate
    """
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

    # Slight extra contrast only for lid detection
    gray = cv2.convertScaleAbs(gray, alpha=1.35, beta=0)

    blur = cv2.GaussianBlur(gray, (3, 3), 0)

    # Vertical edge detector
    sobel = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
    sobel = np.absolute(sobel)

```

```

sobel = np.uint8(sobel)

_, edges = cv2.threshold(sobel, LID_EDGE_THRESHOLD, 255,
cv2.THRESH_BINARY)

h, w = edges.shape

# Search only near the body edges
left_search_x1 = max(0, body_l - LID_SEARCH_MARGIN_PX)
left_search_x2 = min(w, body_l + LID_SEARCH_MARGIN_PX)

right_search_x1 = max(0, body_r - LID_SEARCH_MARGIN_PX)
right_search_x2 = min(w, body_r + LID_SEARCH_MARGIN_PX)

left_band = edges[:, left_search_x1:left_search_x2]
right_band = edges[:, right_search_x1:right_search_x2]

if left_band.size == 0 or right_band.size == 0:
    return None, None, edges

left_strength = np.sum(left_band > 0, axis=0)
right_strength = np.sum(right_band > 0, axis=0)

if np.max(left_strength) < MIN_LID_COLUMN_SIGNAL:
    return None, None, edges

if np.max(right_strength) < MIN_LID_COLUMN_SIGNAL:
    return None, None, edges

# Strong candidate columns only
left_candidates = np.where(left_strength >= 0.65 *
np.max(left_strength))[0]
right_candidates = np.where(right_strength >= 0.65 *
np.max(right_strength))[0]

if len(left_candidates) == 0 or len(right_candidates) == 0:
    return None, None, edges

# Left lid edge = leftmost strong candidate
lid_l = left_search_x1 + int(left_candidates[0])

# Right lid edge = rightmost strong candidate
lid_r = right_search_x1 + int(right_candidates[-1])

if lid_r <= lid_l:
    return None, None, edges

```

```

    return lid_l, lid_r, edges

def classify(body_l, body_r, lid_l, lid_r):
    """
    Convert pixel measurements to inches using the measured
    body width in THIS image.
    Matching testt.py
    """
    body_width_px = body_r - body_l + 1

    px_per_in = body_width_px / BODY_REAL_WIDTH_IN
    in_per_px = 1.0 / px_per_in

    pixel_tolerance = ALIGN_TOLERANCE_IN * px_per_in

    left_offset_px = body_l - lid_l
    right_offset_px = body_r - lid_r

    left_offset_in = left_offset_px * in_per_px
    right_offset_in = right_offset_px * in_per_px

    passed = (
        abs(left_offset_px) <= pixel_tolerance
        and abs(right_offset_px) <= pixel_tolerance
    )

    return {
        "passed": passed,
        "body_width_px": body_width_px,
        "body_width_in": BODY_REAL_WIDTH_IN,
        "px_per_in": px_per_in,
        "in_per_px": in_per_px,
        "pixel_tolerance": pixel_tolerance,
        "left_offset_px": left_offset_px,
        "right_offset_px": right_offset_px,
        "left_offset_in": left_offset_in,
        "right_offset_in": right_offset_in
    }

# =====
# MAIN
# =====

def run_edge_detection():
    """

```

```

Main pipeline:
1. Capture image
2. Preprocess
3. Crop
4. Extract ROIs
5. Detect body edges
6. Detect lid edges
7. Convert px to inches using measured body width
8. Pass/fail decision
"""
image = capture_image_from_webcam(CAMERA_INDEX, WARMUP_TIME)

if SAVE_CAPTURED_IMAGE:
    cv2.imwrite(CAPTURED_IMAGE_PATH, image)

image = preprocess(image)

print("Image shape:", image.shape)

# Crop
crop = image[CROP_Y:CROP_Y + CROP_H, CROP_X:CROP_X + CROP_W]
print("Crop shape:", crop.shape)

if crop.size == 0:
    raise RuntimeError("Crop is empty. Check crop values.")

# ROIs
body_roi = crop[BODY_ROI_Y1:BODY_ROI_Y2, :]
lid_roi = crop[LID_ROI_Y1:LID_ROI_Y2, :]

print("Body ROI shape:", body_roi.shape)
print("Lid ROI shape:", lid_roi.shape)

if body_roi.size == 0 or lid_roi.size == 0:
    raise RuntimeError("Body ROI or lid ROI is empty. Check ROI
values.")

# Detect body
body_l, body_r, body_bin = detect_body_edges(body_roi)

if body_l is None or body_r is None:
    result = {
        "status": "DETECTION_ERROR",
        "passed": False,
        "body_left": None,
        "body_right": None,
        "lid_left": None,

```



```

        "lid_right": None,
        "body_width_px": None,
        "body_width_in": None,
        "px_per_in": None,
        "in_per_px": None,
        "pixel_tolerance": None,
        "left_offset_px": None,
        "right_offset_px": None,
        "left_offset_in": None,
        "right_offset_in": None,
        "align_tolerance_in": ALIGN_TOLERANCE_IN
    }
    print("Vision result:", result)
    return result

# Detect lid
lid_l, lid_r, lid_edges = detect_lid_edges(lid_roi, body_l, body_r)

if lid_l is None or lid_r is None:
    body_width_px = body_r - body_l + 1
    px_per_in = body_width_px / BODY_REAL_WIDTH_IN
    in_per_px = 1.0 / px_per_in
    pixel_tolerance = ALIGN_TOLERANCE_IN * px_per_in

    result = {
        "status": "DETECTION_ERROR",
        "passed": False,
        "body_left": body_l,
        "body_right": body_r,
        "lid_left": None,
        "lid_right": None,
        "body_width_px": body_width_px,
        "body_width_in": BODY_REAL_WIDTH_IN,
        "px_per_in": px_per_in,
        "in_per_px": in_per_px,
        "pixel_tolerance": pixel_tolerance,
        "left_offset_px": None,
        "right_offset_px": None,
        "left_offset_in": None,
        "right_offset_in": None,
        "align_tolerance_in": ALIGN_TOLERANCE_IN
    }
    print("Vision result:", result)
    return result

# Classify
classification = classify(body_l, body_r, lid_l, lid_r)

```

```

result = {
    "status": "PASS" if classification["passed"] else "FAIL",
    "passed": classification["passed"],
    "body_left": body_l,
    "body_right": body_r,
    "lid_left": lid_l,
    "lid_right": lid_r,
    "body_width_px": classification["body_width_px"],
    "body_width_in": classification["body_width_in"],
    "px_per_in": classification["px_per_in"],
    "in_per_px": classification["in_per_px"],
    "pixel_tolerance": classification["pixel_tolerance"],
    "left_offset_px": classification["left_offset_px"],
    "right_offset_px": classification["right_offset_px"],
    "left_offset_in": classification["left_offset_in"],
    "right_offset_in": classification["right_offset_in"],
    "align_tolerance_in": ALIGN_TOLERANCE_IN
}

print("\n==== VISION RESULT =====")
print(f"Status: {result['status']}")
print(f"Body edges: left={body_l}, right={body_r}")
print(f"Lid edges: left={lid_l}, right={lid_r}")
print(f"Measured body width: {result['body_width_px']} px")
print(f"Assumed real body width: {result['body_width_in']:.4f} in")
print(f"Measured scale: {result['px_per_in']:.4f} px/in")
print(f"Measured scale: {result['in_per_px']:.6f} in/px")
print(f"Left offset: {result['left_offset_px']} px")
print(f"({result['left_offset_in']:.4f} in)")
print(f"Right offset: {result['right_offset_px']} px")
print(f"({result['right_offset_in']:.4f} in)")
print(f"Pass/fail tolerance: ±{ALIGN_TOLERANCE_IN:.4f} in")
print(f"(±{result['pixel_tolerance']:.2f} px)")
print("=====\n")

print("Vision result:", result)
return result

if __name__ == "__main__":
    run_edge_detection()

```